

Genie Space Migration with DABs

Author: David Huang

Last updated: February Feb 27, 2026

FEIP:  FEIP-2595: [AI On Databricks] MLOps for AI/BI: Automating Databricks Genie Migra...

Repo: <https://github.com/hiydavid/databricks-genai-examples/tree/main/aibi/genie-migration>

Deprecation Note

As of July 2026, [Declarative Automation Bundles \(DABs\)](#) now supports [Genie Spaces](#). The following **workaround is no longer needed**.

How It Works: Under the Hood

The migration follows a three-phase ETL-like workflow applied to Genie configuration itself:
Export → **Transform** → **Load**.

Phase 1: Export (Source Workspace)

The `export_genie_space` script calls `w.genie.get_space(include_serialized_space=True)` via the Databricks SDK. This returns the full space configuration as a `serialized_space` JSON field — a portable representation containing:

- `config` — sample questions and space settings
- `data_sources` — Unity Catalog table references (`catalog.schema.table`)
- `instructions` — contextual instructions, example SQL, join specs, and SQL snippets
- `benchmarks` — test questions and expected answers

This JSON is written to the workspace file system (`/Workspace/Shared/genie_exports/<title>.json`) and then downloaded locally using the Databricks CLI.

The job runs as a Service Principal with **CAN EDIT** permission on the source Genie space — no human credentials in the pipeline.

Phase 2: Transform (Local / Version Control)

Once the JSON is on the local machine, this is the opportunity to make environment-specific adjustments before promotion:

- **Update catalog/schema references** if the target workspace uses different Unity Catalog names (e.g., `dev_catalog` → `prod_catalog`)
- **Verify warehouse IDs** — the target SQL Warehouse must exist in the destination workspace
- **Review instructions and benchmarks** for any dev-specific content that shouldn't reach production

The modified JSON is then committed to Git, creating a versioned snapshot of the Genie space at the point of promotion. This commit serves as the audit record.

Phase 3: Load (Target Workspace)

The `deploy_genie_space` script handles idempotent deployment to the target workspace:

Scenario	Behavior
No existing space ID provided	Creates a new Genie space; outputs the new space ID
Existing space ID provided	Updates that space in place

On first deployment, save the new space ID — you'll need it for subsequent updates to ensure idempotency.

The deploy job runs as a Service Principal with **CAN USE** on the target SQL Warehouse and **CAN MANAGE** on the target parent folder. The target workspace gets a different `space_id` than the source — this is expected.

Orchestration with DABs

Databricks Asset Bundles serve as the Infrastructure as Code layer that ties all three phases together. The `databricks.yml` configuration defines:

- **Two DAB targets** (`dev` for source, `prod` for target), each with its own workspace URL and Service Principal — enabling separate credential scoping per environment
- **Two Lakeflow Jobs** (`export_genie_space` and `deploy_genie_space`), each running the corresponding Python script as a Databricks notebook
- **Bundle variables** that parameterize the space ID, warehouse ID, parent path, and config file location at deploy time — no hardcoded values

The full migration command sequence looks like:

Source workspace:

```
databricks bundle validate --target dev
databricks bundle deploy --target dev
databricks bundle run export_genie_space --target dev
```

Local:

```
databricks workspace export ../genie_exports/<title>.json --file
./genie_spaces/<filename>.json
git add / commit / push
```

Target workspace:

```
databricks bundle deploy --target prod --var deploy_config_path=...
databricks bundle run deploy_genie_space --target prod
```

See the [project README](#) for the complete command reference, including how to pass `--var target_space_id=<id>` for idempotent updates.

Known Limitations

Limitation	Impact
No conversation history migrated	Conversations are workspace-specific and are not portable
UC permissions not migrated	Unity Catalog grants must be configured separately in the target workspace
Table names deployed as-is	Edit the JSON before committing if catalog/schema names differ between environments
DABs native Genie support pending	This pattern uses public APIs as a workaround; see [GitHub issue #3008](https://github.com/databricks/cli/issues/3008)
UC Functions not migrated	`serialized_space` includes references to SQL functions, not function definitions. In the target workspace, ensure referenced UC functions already exist and required permissions are granted. If source and target share a metastore and those functions are already accessible, recreation is not required.

Getting Started

1. Clone the repo: [databricks-genai-examples/aibi/genie-migration](https://github.com/databricks-genai-examples/aibi/genie-migration)

2. **Configure Service Principals** for both source and target workspaces — see [docs/SETUP_GUIDE.md](#) for detailed provisioning steps
3. **Copy and edit** `databricks.yml.template` → `databricks.yml` with your workspace URLs, SP IDs, and warehouse details
4. **Follow the README** for the complete export and deploy command sequence